Software Architecture · 课程项目

开 题 报 告

Confirmation Report

移动云计算环境下的计算划分系统

—— 以 HyperLPR 车牌识别流水线为研究对象

成员	角色	职责分工
饶晓杰	组长	架构总师；Part C 运行时动态划分决策；答辩主讲
丁奕轩	队员 2	端侧系统改造：源码解耦、端侧基线、可插拔流水线与静态划分联调
陈冠江	队员 3	云端 FastAPI 多切分推理微服务与网络扰动
柯　涵	队员 4	全矩阵基准评测与数据可视化

课题类型　计算划分（Computation Partitioning）系统设计与评测
研究对象　HyperLPR 开源中文车牌识别框架（四阶段推理流水线）
提交日期　2026 年 11 月 3 日
线下汇报　2026 年 11 月 4 日　B7-138

本报告中所有性能数据均由本团队在统一环境下实测获得

目　录

1. 摘要 3
2. 引言 3
2.1. 研究背景 3
2.2. 研究意义 3
2.3. 本报告的范围 3
3. 选定的应用程序：HyperLPR 3
4. 应用程序源码模块结构 4
4.1. 模块到文件的分配结构 4
4.2. 从“固化流水线”到“可插拔流水线” 5
5. 数据流建模与切分点分析 5
6. 性能指标与测量方法 6
7. 实验与结果 6
7.1. 实验环境 6
7.2. 端侧基线（Part A） 6
8. 主要洞察 7
9. 后续工作计划与里程碑 7
10. 结论 8
11. 参考文献 8

1. 摘要

本课题面向移动云计算（Mobile Cloud Computing，MCC）环境，以开源中文车牌识别框架 HyperLPR 为载体，研究**计算划分**（Computation Partitioning）问题：将应用的各计算模块在移动终端与云端之间进行分配，并通过系统实验刻画“划分方式—网络条件—端到端性能”之间的关系。我们首先对 HyperLPR 的源码进行模块级剖析，将其四阶段推理流水线（目标定位 → 几何对齐 → 字符识别 → 属性分类）解耦为四个可独立调度的算子，并建立“图像 × 设置”的可复现测量方法。在统一环境下完成的端侧基线测量表明：四阶段合计约 92 ms，峰值内存 142.6 MB；基线同时暴露出推理引擎默认多线程会占满整机 CPU（约 31 等效核心），线程数与单核时延的对应关系是移动端部署必须回答的问题，已列入测量计划。端侧仅需约 19 ms 完成定位与对齐，即可把上传载荷从数百 KB 压缩至 9.75 KB（削减 97.5%）。以上结果划定了后续划分方案设计的搜索空间：占端侧算力约 77% 的识别阶段是卸载的候选对象，而上传载荷的量级由切分点位置决定。划分方案的设计与多划分方式的性能对比，将按课程里程碑在中期报告（2026-11-29）中给出。

2. 引言

2.1. 研究背景

移动云计算中的**计算划分**，指把应用软件分解为一组模块，并决定哪些模块在本地执行、哪些部分卸载（offload）到远端服务器或云端。划分点选在哪里，直接决定三件事：

1. **端到端时延** = 端侧计算 + 上行传输 + 云侧计算 + 下行传输；
2. **网络流量**（流量成本与弱网可用性）；
3. **隐私暴露面**（上传完整原图 vs 仅上传局部切片）。

静态划分（离线固定一个切分点）在网络稳定时有效，但真实移动网络会在 5G / WiFi / 4G / 弱网之间反复跳变。同一静态切分点在好网与坏网下的表现可以相差一个数量级，因此课程 Part C 进一步要求实现**运行时动态自适应划分**。而动态决策的前提，是先把应用拆成可独立调度的算子，并拿到每个算子的真实耗时与数据载荷（Cost Profiling）——这正是本课题 Part A 的工作，也是本报告的主体。

2.2. 研究意义

车牌识别是典型的**计算密集且时延敏感**（compute-intensive & latency-sensitive）的移动视觉应用，广泛出现在智慧停车、辅助驾驶、移动执法等场景。以它为载体的划分研究具有双重意义：方法上，它是一条阶段清晰、中间张量存在数量级体积落差的流水线，划分收益显著且易于量化；应用上，“端侧做轻量预处理 + 云端做重推理”的模式可直接迁移到人脸识别、手势识别、OCR 等同类移动视觉任务。

2.3. 本报告的范围

按课程里程碑安排，本开题报告重点回答两个问题：**选定什么应用**（第 2 章）与**应用源码的模块结构是什么**（第 3、4 章），并给出端侧基线的初步测量结果（第 6 章）。各计算模块在端云之间如何划分、不同划分方式的性能对比，属于中期报告的交付内容，本报告仅在数据流分析中给出切分点的候选位置（第 4 章），在后续工作计划中列出推进路径（第 8 章）。

3. 选定的应用程序：HyperLPR

定位：开源高性能中文车牌识别框架，支持单行蓝牌 / 黄牌、新能源绿牌、教练车、双层黄牌等主流车牌类型。

技术栈：Python 侧基于 ONNXRuntime（推理引擎）+ OpenCV（图像处理）；C++ / 移动端侧基于 MNN 引擎，同一套模型双后端部署，贴近真实移动场景。

选型理由：

- 天然是一条**四阶段顺序流水线**（定位 → 对齐 → 识别 → 分类），切分点语义清晰；
- 中间张量存在**数量级的体积落差**（原图 MB 级 → 车牌切片 KB 级），划分收益显著且易于量化；
- **纯 CPU 可跑**（1080p 全流程约 92 ms），端侧与云侧可在同一台机器上等价复现，实验变量可控；
- 跨平台（Linux / macOS / Android / iOS），代码规模适中（核心包 1697 行），适合做源码级手术。

4. 应用程序源码模块结构

本章是开题报告的核心交付。我们从**模块到文件的分配结构**（Module-to-File Allocation）与**运行时调用结构**两个视角剖析源码。

4.1. 模块到文件的分配结构

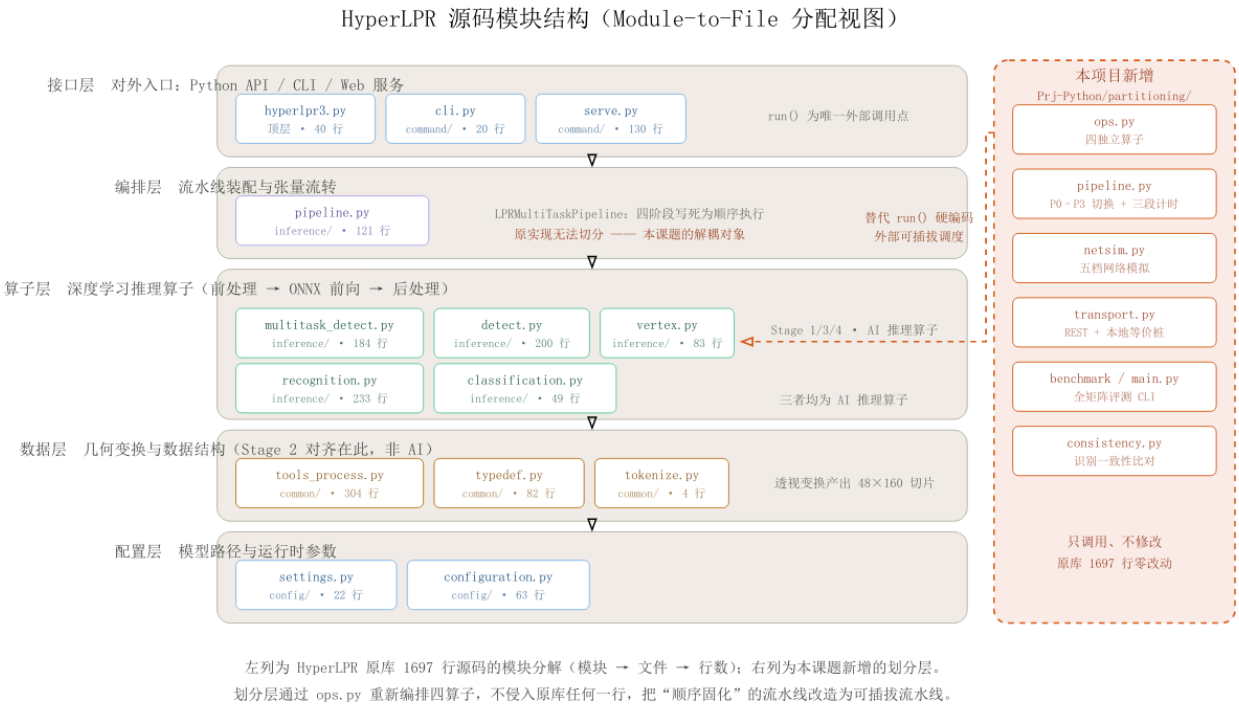


图 1　图 1　HyperLPR 源码模块结构（左：原库 1697 行的分层分解；右：本课题新增的划分层）

原库可分为五个层次，模块与源文件的对应关系如下表（框内数字为源文件行数）：

层次	职责	关键源文件	行数
接口层	对外暴露识别入口与 CLI / Web 服务	`hyperlpr3.py`、`command/cli.py`、`command/serve.py`	190
编排层	把四个算子串成顺序流水线，负责张量与元数据流转	`inference/pipeline.py`（`LPRMultiTaskPipeline`）	121
算子层	检测 / 识别 / 分类的 ONNX 会话封装（前处理 → 推理 → 后处理）	`multitask_detect.py`、`detect.py`、`vertex.py`、`recognition.py`、`classification.py`	749
数据层	图像几何变换（对齐）、类型定义、字符词表	`common/tools_process.py`、`common/typedef.py`	386
配置层	模型路径、运行时参数与初始化	`config/settings.py`、`config/configuration.py`	85

值得注意的是，**四角点由检测阶段一并预测**（YOLOv5 风格多任务头，每锚点输出 15 维），因此“检测 + 对齐”天然可以打包留在端侧，无需为对齐单独跑一个模型；而 Stage 4 分类并非每帧必跑——只有当识别出的车牌号长度 ≥ 7 且规则判不出类型时才触发，属于库内既有的算力优化。

4.2. 从“固化流水线”到“可插拔流水线”

原库把四个阶段写死在 `LPRMultiTaskPipeline.run()` 一个方法内：它要么全本地执行、要么整体外移，内部没有“能插拔”的接缝，无法表达“检测在端侧、识别在云端”这类分工。Part A 的关键手术就是把 `run()` 拆成四个可单独调用的算子（新增 `partitioning/ops.py`，**不修改**原库任何一行），使划分决策从“代码结构”上升为“运行时配置”。

阶段	实现类 / 函数	输入 → 输出	算法性质
Stage 1 定位	`MultiTaskDetectorORT`	原图 → 每车 14 维（框 4 + 置信 1 + 四角点 8 + 层标志 1）	深度学习（YOLOv5 风格多任务头）
Stage 2 对齐	`get_rotate_crop_image`	原图 + 四角点 → 切片 48×160×3	传统 CV（透视变换，纯几何）
Stage 3 识别	`PPRCNNRecognitionORT`	切片 → 车牌号字符串 + 置信度	深度学习（CRNN + CTC 解码）
Stage 4 分类	`ClassificationORT`	切片 → 类别索引（蓝 / 黄 / 绿）	深度学习（3 类 CNN，条件触发）

5. 数据流建模与切分点分析

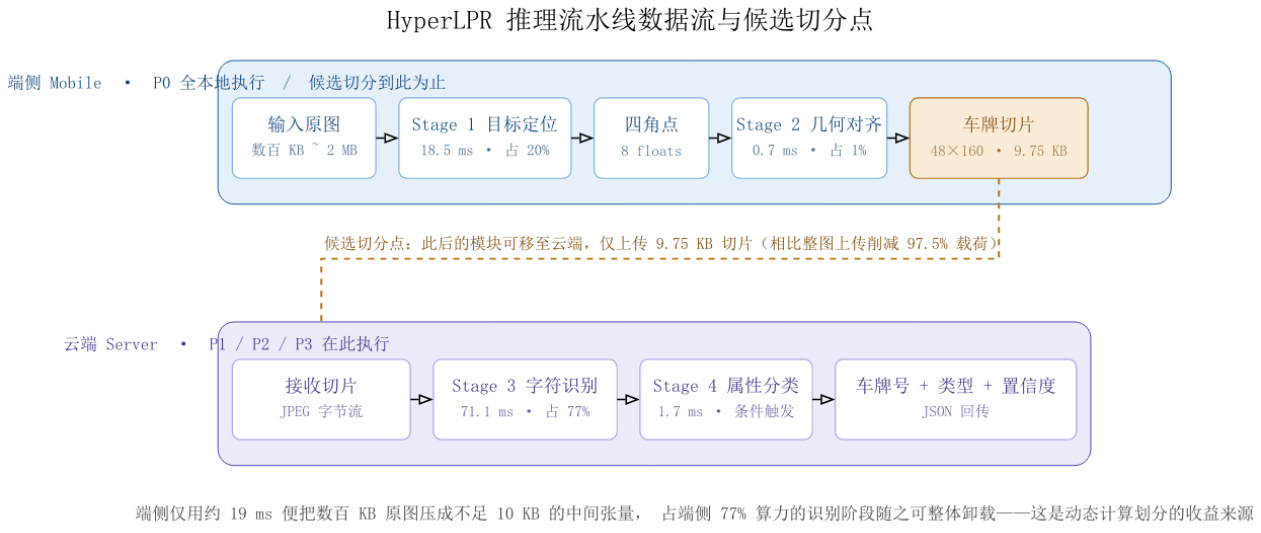


图 2　图 2　推理流水线数据流与候选切分点（节点标注为实测耗时占比与载荷）

各节点张量形态与载荷的定量对比（完整表见仓库 `doc/tensor_shapes.md`）：

节点	张量形态	载荷	说明
输入原图	H×W×3，uint8 BGR	385 KB 2 MB	1080p 典型 1920×1080
检测输出	1×25200×15，float32	—	经置信度过滤 + NMS 后每车剩 14 维

节点	张量形态	载荷	说明
四角点	4×2，int	—	检测阶段“白送”，无需额外模型
车牌切片	**48×160×3，uint8**	**9.75 KB**	**JPEG q95 编码；切分点产物**
识别输出	1×40×6625，float32	—	40 时间步 × 6625 类字符，CTC 解码

端侧只需付出约 19 ms 完成定位与对齐，即可把上传载荷从数百 KB 压到不足 10 KB；而占端侧 77% 算力的 Stage 3，其输入恰恰只是这张 9.75 KB 切片而非原图——卸载它既不增加传输量，又省掉了最贵的算力。这使**对齐之后**成为最有潜力的候选切分点，其收益的具体量化将在中期报告的划分实验中验证。

关于早期数据的修正：仓库 dev 分支早期文档记录“Detector 19.2 ms、占 48%”，与本项目实测（18.54 ms、占 20.1%）不符。本报告全部采用实测口径。这一修正实际上**强化**了“对齐后切分”的合理性：端侧付出的代价比原先认为的更小。

6. 性能指标与测量方法

按照课程对 Cost Profiling 的要求，我们选取以下指标并给出测量方法：

指标	定义	测量方法
阶段执行时间	四阶段算子各自的墙钟耗时	`time.perf_counter()` 逐算子独立计时，20 轮取均值与标准差
内存占用	进程峰值常驻内存 RSS	`psutil` 采样 (缺失时自动降级为只测时间)
CPU 占用	进程消耗的 CPU 时间折算的**等效核心数**（1.0 = 跑满一核）	`time.process_time()` 差值 ÷ 墙钟时间，另折算整机百分比供参照
网络载荷	各切分候选点的上传字节数	实际编码产物 `len(bytes)` 而非估算

复现性：全部指标由 `profile_baseline.py` 支持 `--image / --rounds / --csv` 在任意机器复跑；对同一输入重复多次取均值与标准差以抑制抖动。端到端时延、传输时延等涉及网络条件的指标，将随划分方案一同在中期报告定义与测量。

7. 实验与结果

7.1. 实验环境

WSL Ubuntu（共享 32 核 x86_64 CPU）/ Python 3.12 / ONNXRuntime（CPU 执行）/ OpenCV；模型为 `hyperlpr3` 自带的 4 个 ONNX 权重（检测 y5fu_320x/640x、识别 rpv3_mdict_160、分类 litemodel_cls_96x）。测试图像为仓库自带 7 张（含 1920×1080 自然场景图）。

7.2. 端侧基线（Part A）

输入 `test_img.jpg`（1920×1080），逐阶段独立计时 20 轮取均值：

阶段	均值 (ms)	标准差	端侧算力占比	是否 AI 推理
Stage 1 目标定位	18.54	±5.65	20.1%	是
Stage 2 几何对齐	0.67	±0.14	0.7%	否（几何）
Stage 3 字符识别	71.09	±25.99	77.2%	是
Stage 4 属性分类	1.74	±2.19	1.9%	是（条件触发）

阶段	均值 (ms)	标准差	端侧算力占比	是否 AI 推理
端侧合计	**92.04**	—	**100%**	—

系统资源占用与线程语义：峰值常驻内存（RSS）**142.6 MB**，内存维度是轻量负载。CPU 维度的测量结果为 **31.0 等效核心**（折合 32 核整机约 97%）——这是 ONNXRuntime 默认多线程（intra_op = 全核）调度的结果，**不是**早期表述中的“约单核跑满”（该说法源于把整机百分比误读为单核占比，本报告已按“等效核心数”口径修正）。这一发现对移动端部署有直接约束：真实手机的核数远少于实验机，必须显式限制推理引擎线程数并重测时延，线程数与端侧耗时的对应关系已列入第 8 章测量计划。

可复现性与测量波动：基线脚本 `profile_baseline.py` 支持 `--image / --rounds / --csv`，可在任意机器复跑；连续三次独立运行（每次 20 轮）端侧合计为 89.93 / 94.55 / 104.86 ms（±8%），而 RSS（142.7–144.3 MB）、等效核心数（30.9–31.5）与切片载荷（9.75 KB）三项资源指标高度稳定。耗时波动主要来自 WSL 共享 CPU 的调度噪声，故表中耗时应作**同量级基准**引用，终期前将在物理机复测对照。

8. 主要洞察

除实验数字本身，我们希望传达两点超出“跑得快”的观察：

1. **切分点的本质是“张量体积的悬崖”。**数据流 DAG 上存在天然的不连续点——对齐之后载荷从 MB 级跳到 KB 级。找到这个悬崖，比在连续代价曲线上做数值优化更稳健；而悬崖位置由**算法结构**（角点由检测白送、对齐产出规整小图）决定，与硬件无关，因此结论可迁移到其他视觉流水线。
2. **“是否 AI”不影响切分决策，“单位传输收益”才影响。**对齐（0.7 ms）虽不是 AI，却是潜在方案的枢纽——它用不到 1% 的算力换来切片与原图的解耦。划分决策应关注每个算子的“耗时/载荷压缩比”，而非其算法属性。这一量化视角将直接用于中期报告的划分方案设计与评测。

9. 后续工作计划与里程碑

序	工作内容	负责人 / 时间
1	扩充测试集：补 20+ 张真实场景图（不同分辨率 / 光照 / 角度）后重跑全矩阵	队员 2、4 / 11 月中
2	云端联调：队员 3 的 FastAPI 微服务就绪后，接入真实云端服务复跑同一矩阵，替换本地模拟的云侧数据	队员 2、3 / 11 月中
3	Part C 动态划分：NetworkMonitor + PolicySelector + 运行时热切换，扰动实验验证收敛性	组长 / 11 月下
4	线程数对照：限制 ONNXRuntime `intra_op` = 1 / 2 / 4 / 8 复测端侧基线，确定移动端部署的线程配置与真实单核时延	队员 2 / 11 月上
5	物理机复测对照，消除 WSL 虚拟化噪声；能耗代理指标测量	队员 2、4 / 12 月上
6	终期报告与 6 分钟演示演练（含演示清单，预演 ≥ 10 次）	全组 / 12 月中

里程碑	内容	时间
M1 开题报告	选定应用 + 源码模块结构 + 端侧基线（本报告）	2026-11-03 提交 / 11-04 汇报

里程碑	内容	时间
M2 中期报告	不同划分方式及其性能的完整对比	2026-11-29
M3 终期报告	引言 / 正文 / 实验 / 结论 / 参考文献的完整报告 + 源码	2026-12-21
M4 现场答辩	6 分钟系统演示：划分方案对性能的影响与动态划分实现	2026-12-23

10. 结论

本报告完成了计算划分课题的开题工作：选定 HyperLPR 作为研究对象，完成其源码的模块级剖析与四算子解耦，建立了“图像 × 设置”的可复现测量方法，并拿到了端侧基线的第一批实测数据——四阶段合计约 92 ms、峰值内存 142.6 MB，端侧以约 19 ms 的代价把潜在上传载荷从数百 KB 压缩到 9.75 KB（削减 97.5%）。基线同时给出两个对后续工作有约束力的事实：占端侧算力约 77% 的识别阶段是卸载收益最大的候选；推理引擎默认多线程会占满整机 CPU（约 31 等效核心），移动端部署必须显式控制线程数。按课程里程碑，各划分方式的设计与性能对比、识别一致性验证将在 11 月 29 日中期报告中给出，动态自适应划分与系统演示在终期完成。

11. 参考文献

1. Cuervo E., Balasubramanian A., Cho D., et al. MAUI: Making Smartphones Last Longer with Code Offload. **Proc. MobiSys**, 2010.
2. Chun B., Ihm S., Maniatis P., et al. CloneCloud: Elastic Execution between Mobile Device and Cloud. **Proc. EuroSys**, 2011.
3. Khan A. U. R., Othman M., Madani S. A., et al. A Survey of Mobile Cloud Computing Application Models. **IEEE Communications Surveys & Tutorials**, 2014.
4. HyperLPR: 开源高性能中文车牌识别框架. GitHub: szad670401/HyperLPR（本团队工作仓库: ya2yo/HyperLPR）.
5. Redmon J., Divvala S., Girshick R., et al. You Only Look Once: Unified, Real-Time Object Detection. **Proc. CVPR**, 2016.
6. Shi B., Bai X., Yao C. An End-to-End Trainable Neural Network for Image-based Sequence Recognition and Its Application to Scene Text. **IEEE TPAMI**, 2017.
7. Flinn J. Balancing Performance, Energy, and Quality in Pervasive Computing. **Proc. ICDCS**, 2002.